

```

1827     grid_times = np.append(grid_times, phases[-1]["t1"])
1828     grid_outages = 0
1829     grid_min_margin = float("inf")
1830     for t in grid_times:
1831         pos = position_at(phases, float(t))
1832         direct = link_state(dem, pos, gateway, thresholds["transport_gateway"], freq, obs)
1833         margins = [float(direct["margin_db"])] if direct["available"] else []
1834         for point, back in zip(relay_points, backhaul):
1835             access = link_state(dem, pos, point, thresholds["transport_relay"], freq, obs)
1836             if access["available"]:
1837                 margins.append(min(float(access["margin_db"]), float(back["margin_lower_db"])))
1838         if not margins:
1839             grid_outages += 1
1840         else:
1841             grid_min_margin = min(grid_min_margin, max(margins))
1842     unresolved = [x for x in leaves if x["mode"] == "unresolved"]
1843     return {
1844         "leaf_intervals": len(leaves),
1845         "segments": segments,
1846         "unresolved_intervals": len(unresolved),
1847         "outage_intervals": len(unresolved),
1848         "min_margin_lower_db": min(
1849             (x["margin_lower_db"] for x in leaves if x["mode"] != "unresolved"),
1850             default=-1e9,
1851         ),
1852         "max_unresolved_interval_s": max((x["t1"] - x["t0"] for x in unresolved), default=0.0),
1853         "grid_step_s": grid_step_s,
1854         "grid_samples": len(grid_times),
1855         "grid_outages": grid_outages,
1856         "grid_min_margin_db": grid_min_margin if math.isfinite(grid_min_margin) else -1e9,
1857     }

```

代码清单9明确列出离线搜索后固化的六个悬停点和 W1-W7 七个波次，并展示中继实体机、能源组件和运输资源状态的初始化。这一清单同时限定了正文第三问的复现边界：输入必须是所选问题二的 22 架次方案。

Listing 9 问题三固定点位、波次与资源分配起始逻辑

```

1869 def _schedule_q3_joint(
1870     data: dict[str, Any], q2_trips: list[dict[str, Any]]
1871 ) -> tuple[list[dict[str, Any]], list[dict[str, Any]], list[dict[str, Any]], dict[str, Any]]:
1872     """Wave-based joint transport/relay schedule with event-level resource accounting."""
1873
1874     # Deterministic points obtained by search_relay_pair.py on 5 s blackout samples.
1875     # Every selected point is re-certified below, and every complete trajectory is
1876     # subsequently subjected to the continuous interval proof.
1877     points = {
1878         "P1A": (109.28277777777778, 23.026944444444446, 649.4981079101562),
1879         "P1B": (109.22277777777778, 23.040277777777778, 587.5166015625),
1880         "P2A": (109.20555555555556, 23.049444444444443, 703.3844604492188),
1881         "P2B": (109.22555555555556, 23.069444444444446, 626.3524169921875),
1882         "P3A": (109.23833333333333, 23.052777777777777, 534.8773498535156),
1883         # W4 replacement for P1B. This nearby DEM point closes the short
1884         # conservative-certificate gap on T017 while preserving T016 coverage.
1885         "P4B": (109.20333333333333, 23.020833333333332, 689.1195983886719),
1886     }
1887     if len(q2_trips) != 22:
1888         raise ValueError(
1889             f"Q3 relay wave plan is bound to the selected 22-trip Q2 solution, got {len(q2_trips)}"
1890         )
1891     wave_definitions = [
1892         ("W1", list(range(0, 8)), ["P1A", "P1B"]),
1893         ("W2", list(range(8, 12)), ["P2A", "P2B"]),
1894         ("W3", list(range(12, 15)), ["P3A"]),
1895         ("W4", list(range(15, 17)), ["P1A", "P4B"]),
1896         ("W5", list(range(17, 19)), ["P2B"]),
1897         ("W6", list(range(19, 21)), ["P1A", "P2A"]),
1898         ("W7", [21], ["P2A"]),
1899     ]
1900     boxes, dem, nodes, types = data["boxes"], data["dem"], data["nodes"], data["transport_types"]
1901     depot = nodes["001"]
1902     unit_ready = {u["unit_id"]: 0.0 for u in data["transport_units"]}
1903     battery_ready = {
1904         f"{type_id}-BAT-{index:02d}": 0.0
1905         for type_id, info in data["battery_inventory"].items()

```